Modelling: Micro vs macro

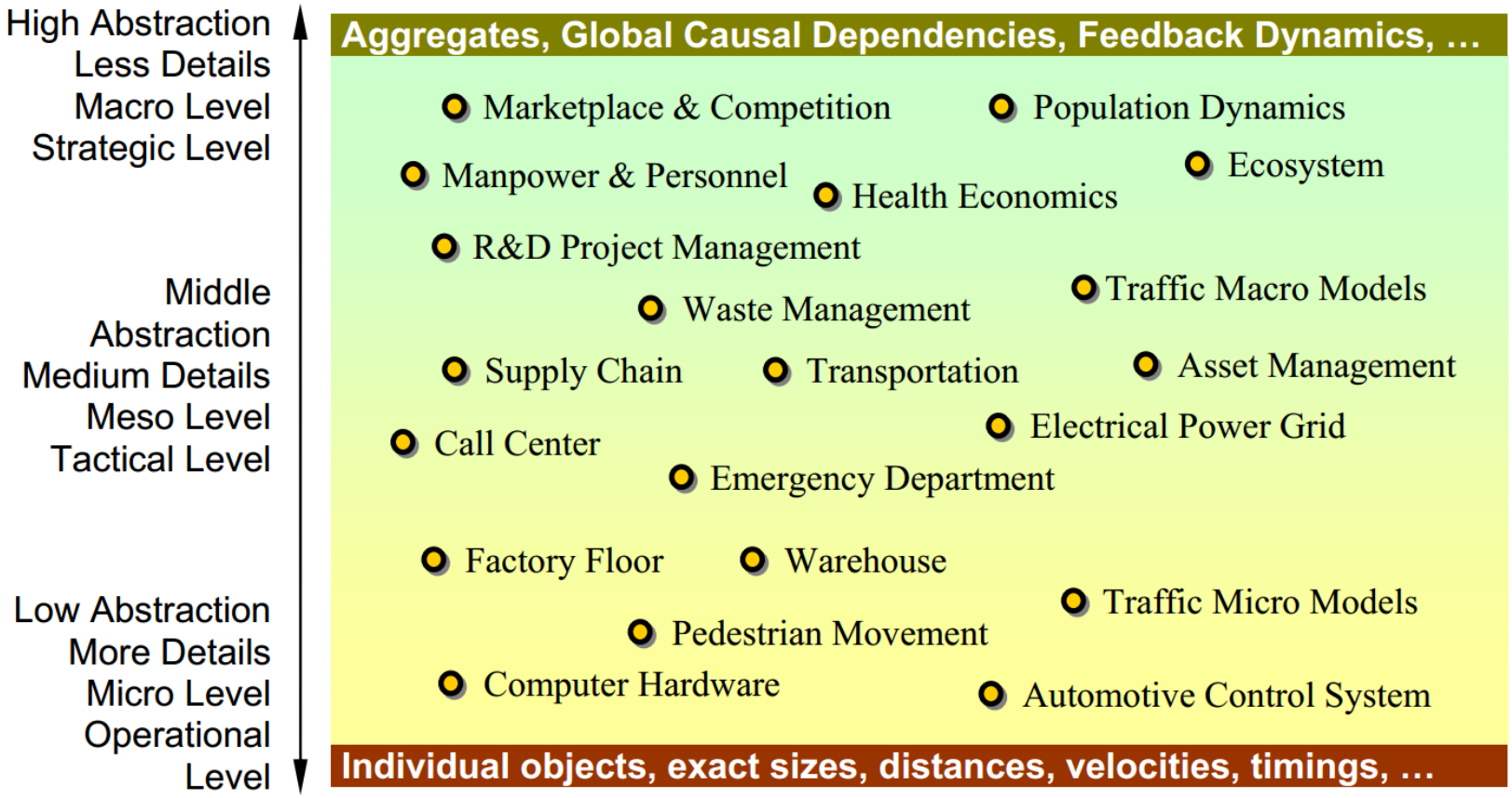


Figure 2: Applications of Simulation Modeling on Abstraction Level Scale

A. Borshchev and A. Filippov. From System Dynamics and Discrete Event to Practical Agent Based Modeling: Reasons, Techniques, Tools. The 22nd International Conference of the System Dynamics Society, July 25 - 29, 2004, Oxford, England

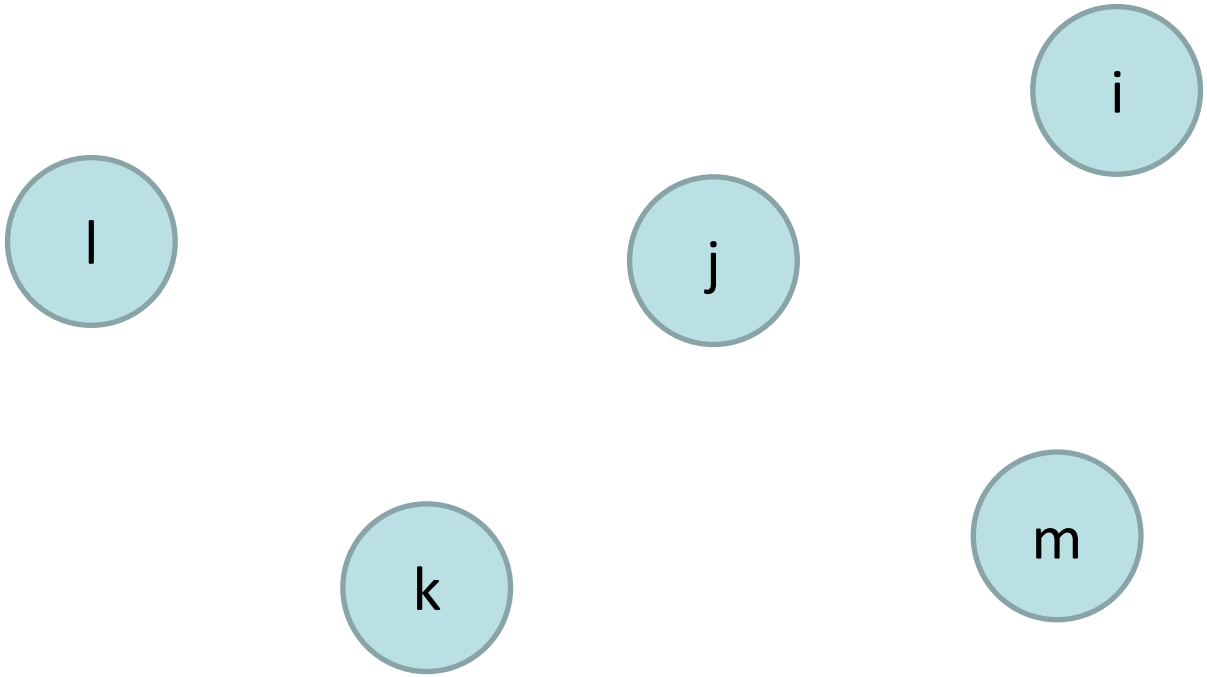
Agent-based modelling (ABM)

Agent-based modelling does the following:

- It disaggregates the problem into agents (e.g. human individuals, animals, households, etc);
- models the system and let agents influence each other;
- lets system evolve in a bottom-up way, and see which emergent properties appear on a systems scale (micro->macro link)
- Sometimes these systems feed back and influence individual agents (macro->micro link)

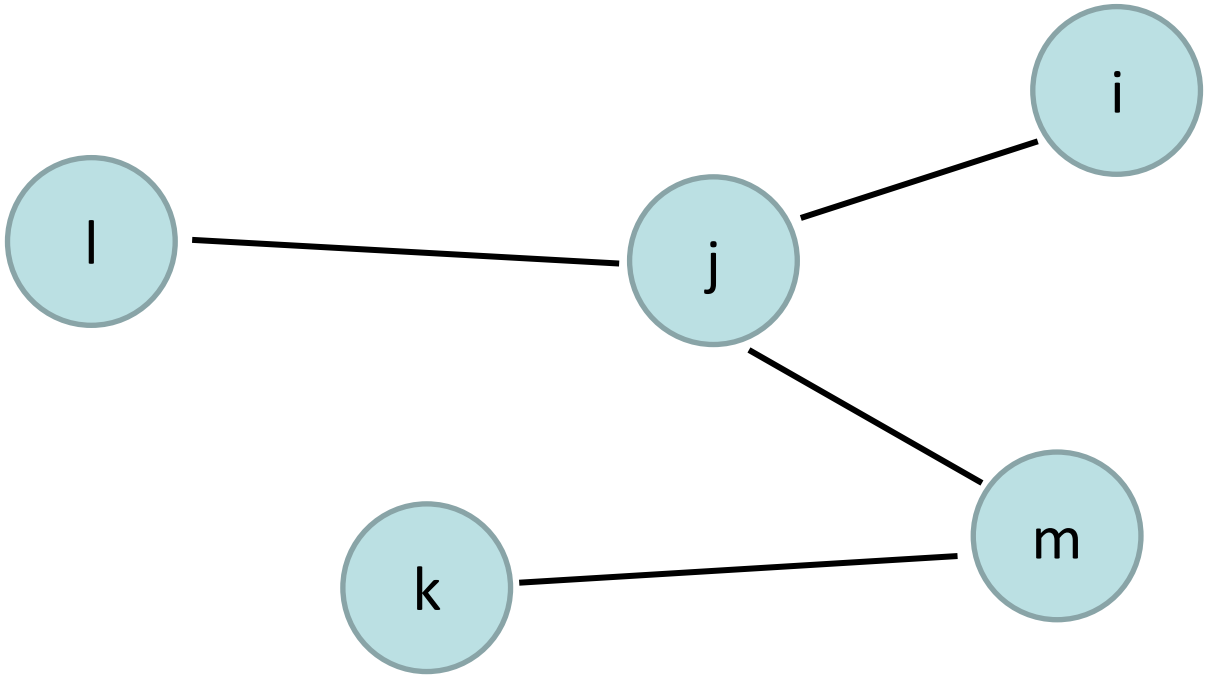
What is agent based modelling?

Step 1: Define your *autonomous* agents (e.g. persons, vehicles, households, ants, robots, companies, …)



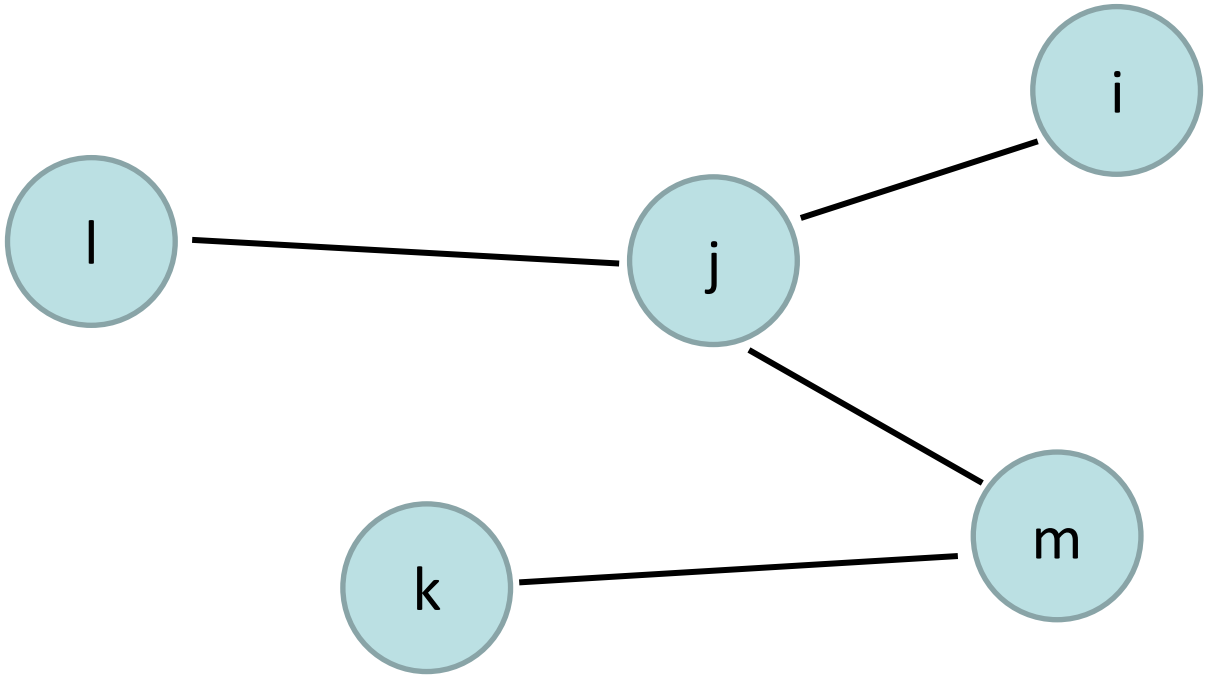
What is agent based modelling?

Step 2: Define interactions (agent-agent, and agent-environment)



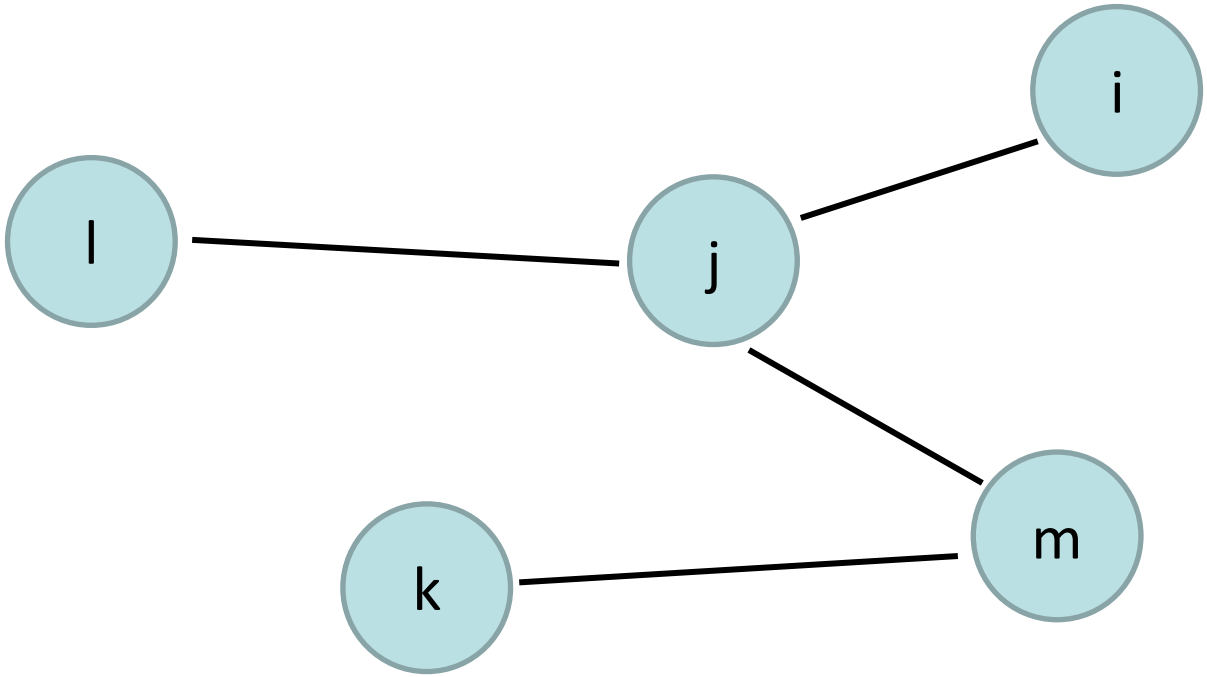
What is agent based modelling?

Step 3: Iterate over time

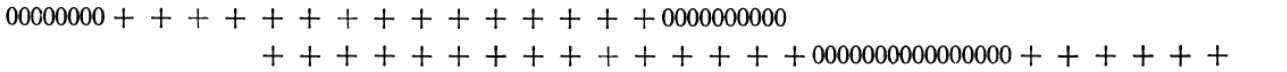


What is agent based modelling?

That's it!



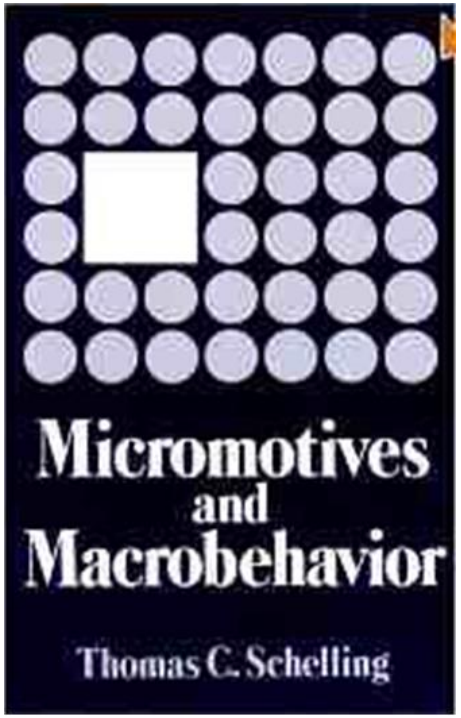
Segregation



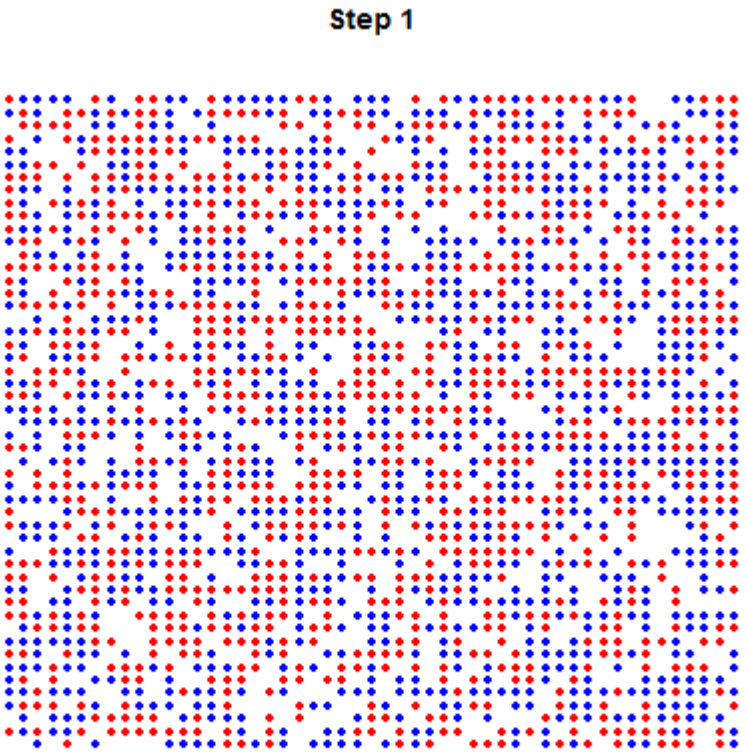
Schelling TC (1969)

Thomas Schelling (1971) proposed a simple model of ethnical segregation. The model consists of people from two ‘classes’, randomly distributed onto a chessboard.

A person will move if he/she is surrounded by less than 1/3 of people from his/her own ‘class’

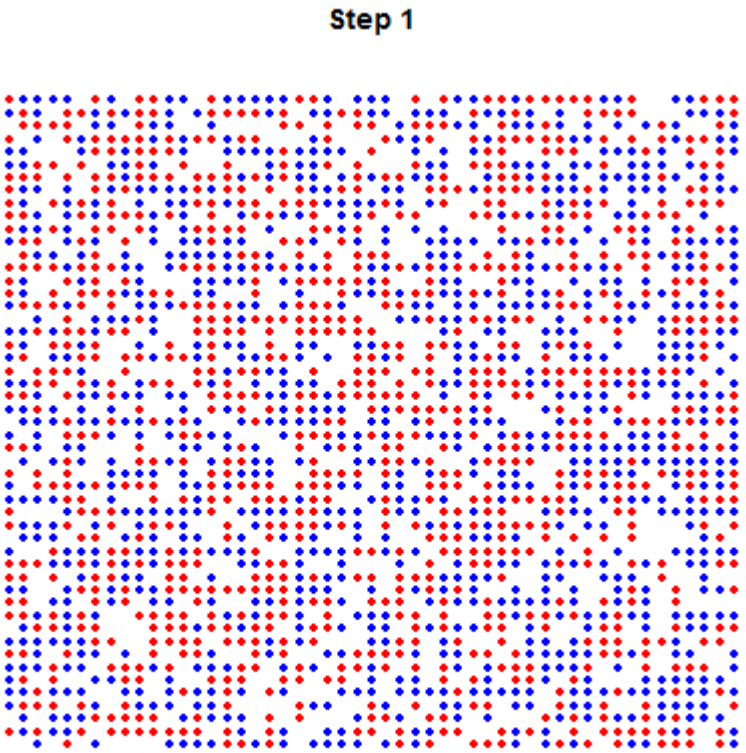


Segregation simulation



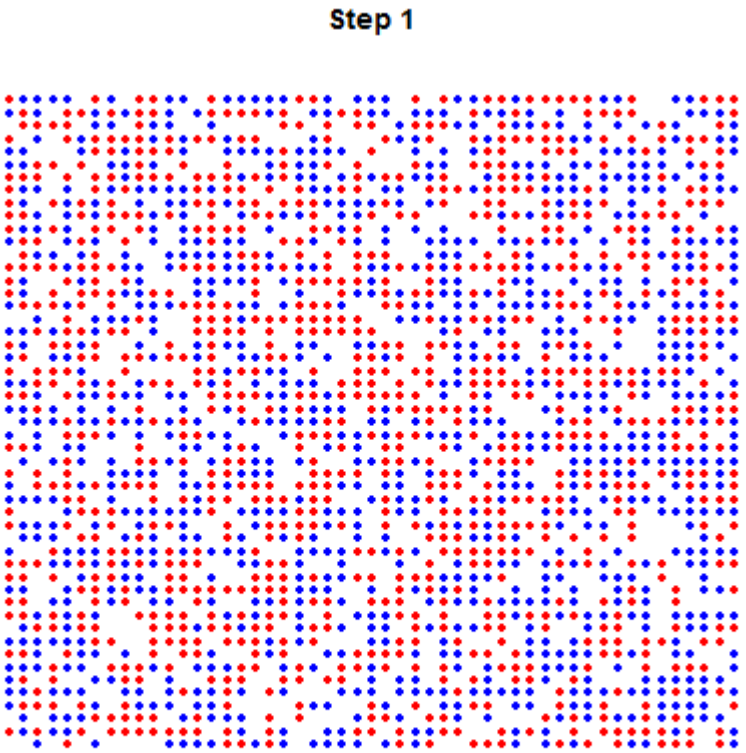
http://rsnippets.blogspot.co.uk/2012/04/

Segregation simulation



http://rsnippets.blogspot.co.uk/2012/04/

Segregation simulation



http://rsnippets.blogspot.co.uk/2012/04/

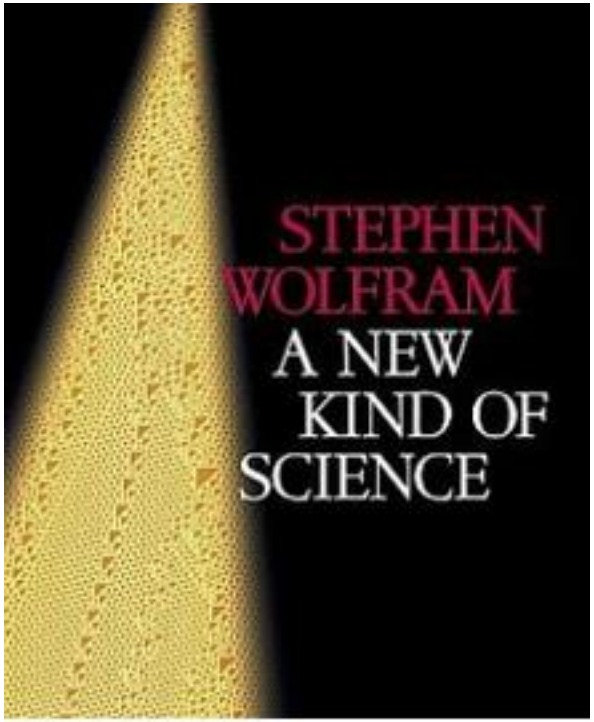
What are the policy implications?

Cellular Automata

A cellular automaton is a rule, defining how the state of a cell in a grid is updated, depending on the states of its neighbour cells.

Cellular-automata simulations are discrete both in time and space.

In their simplest form, CA does not strictly fall into the class of Agent Based Models (ABMs), but are often used for spatial ABMs

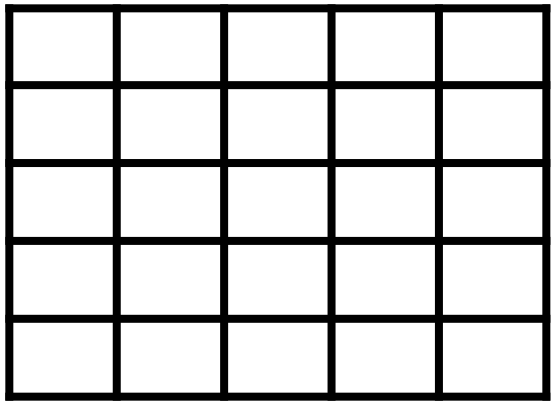


Cellular Automata

The grid can have an arbitrary number of dimensions:



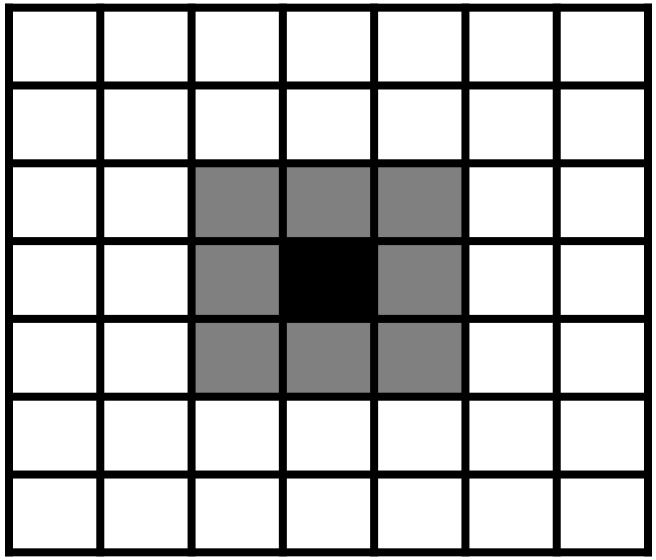
1-dimensional
cellular automaton



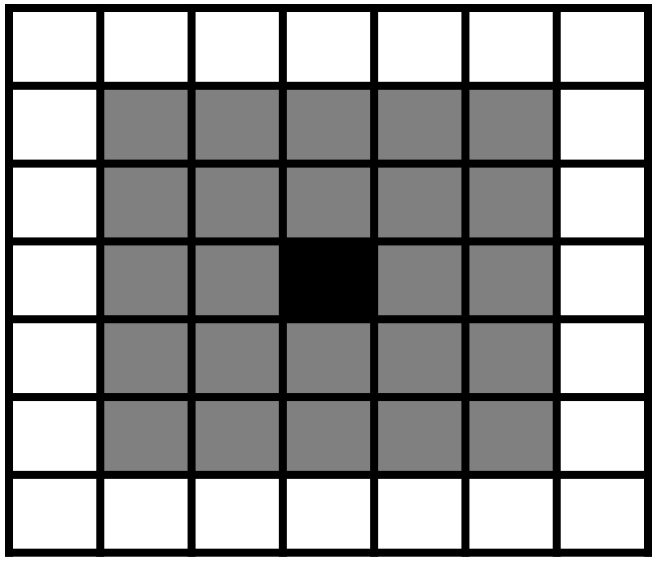
2-dimensional
cellular automaton

Moore neighbourhood

The cells are interacting with each neighbour cell, and the neighbourhood can be defined in different ways, e.g. the *Moore neighbourhood*:



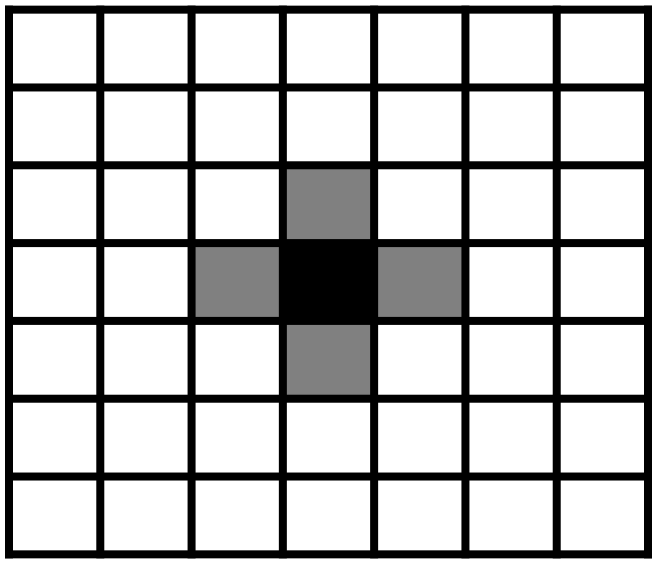
1st order Moore neighbourhood



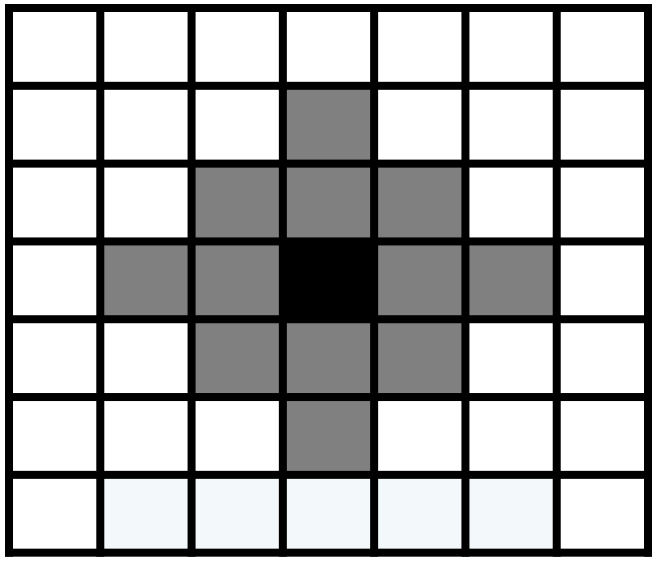
2nd order Moore neighbourhood

Von-Neumann neighbourhood

The cells are interacting with each neighbour cells, and the neighbourhood can be defined in different ways, e.g. the *Von-Neumann neighbourhood*:



1st order Von-Neumann neighbourhood



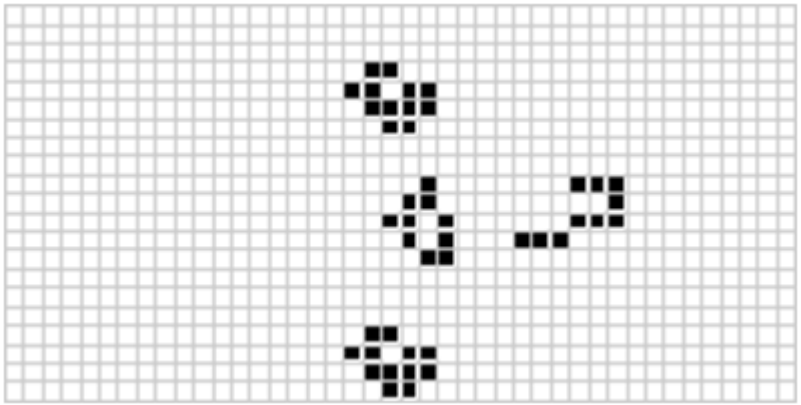
2nd order Von-Neumann neighbourhood

Game of Life

Ni = Number of 1st order Moore neighbours to cell i that are activated.

For each cell i:

- Deactivate: If Ni <2 or Ni >3.
- Activate: If Ni =3



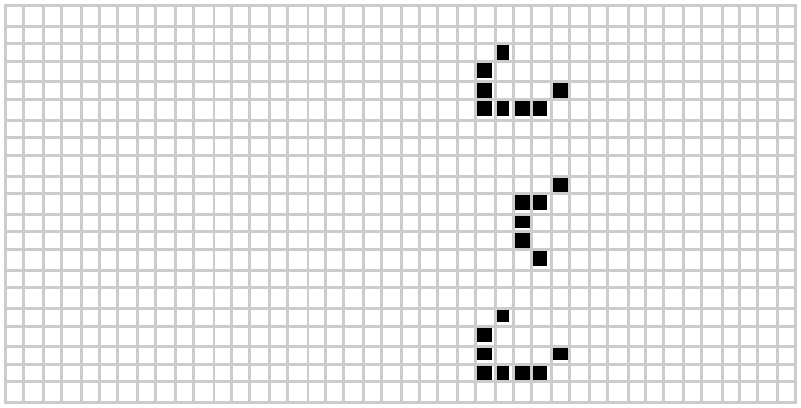
www.mathworld.wolfram.com

Game of Life

Ni = Number of 1st order Moore neighbours to cell i that are activated.

For each cell i:

- Deactivate: If Ni <2 or Ni >3.
- Activate: If Ni =3



www.mathworld.wolfram.com

Spread of disease

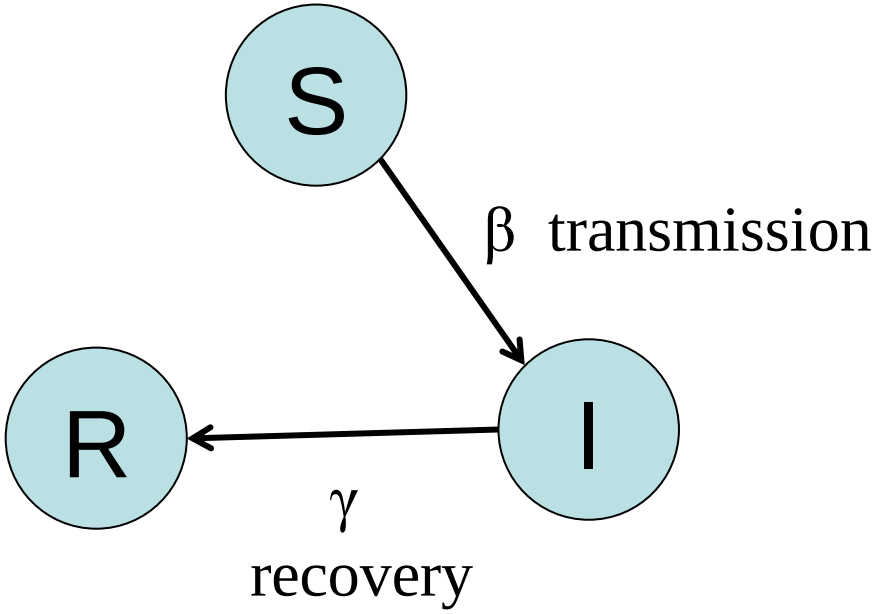
Next example of agent based modelling:

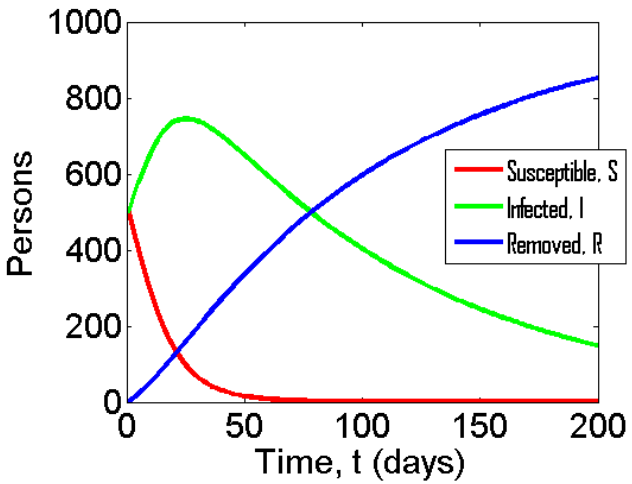


Kermack-McKendrick model

The Kermack-McKendrick model is specified as:

S: Susceptible people
I: Infected people
R: Removed (immune) people
β: Infection rate
γ: Immunity rate





$$\frac{dS}{dt} = -\beta\, I(t)S(t)$$

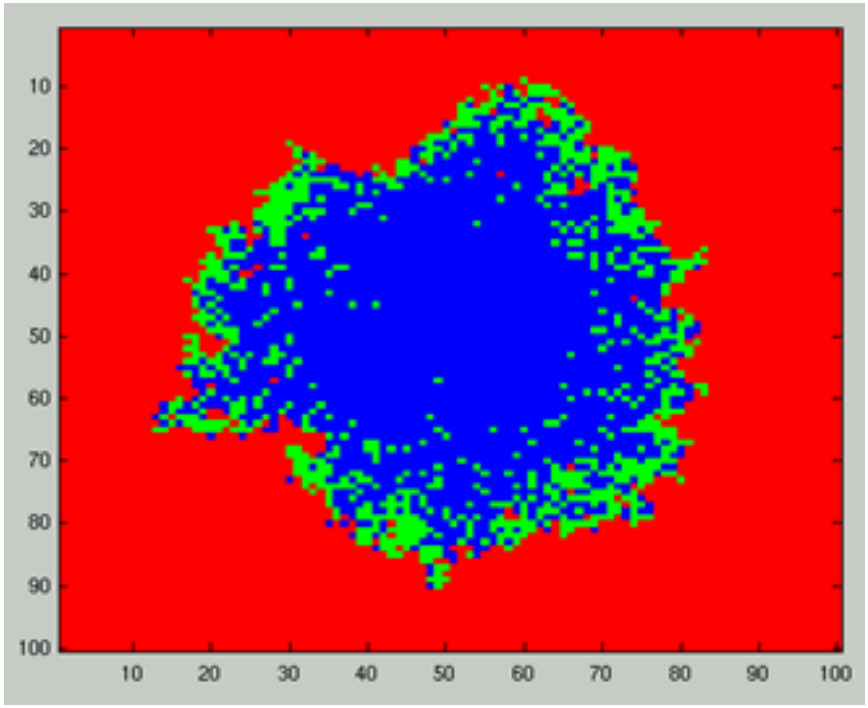
$$\frac{dI}{dt} = \beta\, I(t)S(t) - \gamma\, I(t)$$

$$\frac{dR}{dt} = \gamma\, I(t)$$

Kermack-McKendrick model

The Kermack-McKendrick model is specified as:

S: Susceptible persons
I: Infected persons
R: Removed (immune) persons
β: Infection rate
γ: Immunity rate

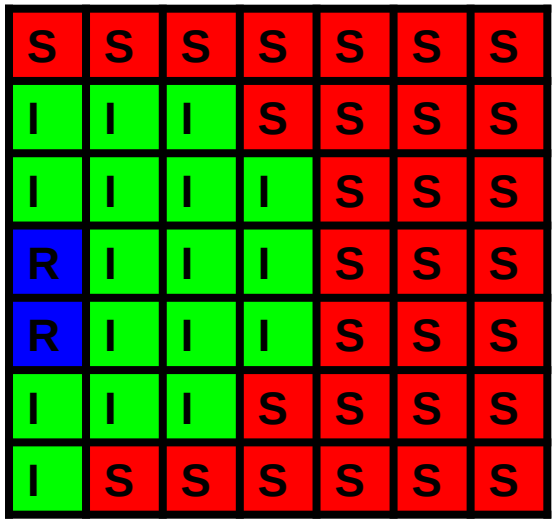


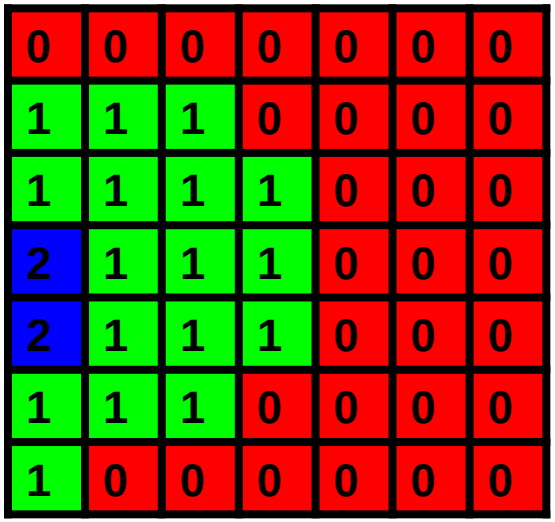
Kermack-McKendrick model

The Kermack-McKendrick model is specified as:

For the MATLAB implementation, we need to decode the states {S, I, R}={0, 1, 2} in a matrix x.

–





Cellular-automaton implementation

Implementation of a 2-dimensional cellular-automaton model in MATLAB is done like this:

Iterate the time variable, t

Iterate over all cells, i=1..N, j=1..N

Iterate over all neighbors, k=1..M

The iteration over the cells can be done either *sequentially,* or *randomly*.

Set parameter values

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

Define grid, x

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));   % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

Define neighborhood

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

Main loop. Iterate the time variable, t

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];
```

```
% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

Iterate over all cells, i=1..N, j=1..N

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

For each cell i, j: Iterate over the neighbors

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

end
```

The model, i.e. updating rule go here.

```
% set parameter values
N=100;                  % Grid size (NxN)
beta=0.01;              % Infection rate
gamma=0.01;             % Immunity rate

% define grid
x = round(rand(N,N));    % The grid x, is coded as:  0=Susceptible, 1=Infected, 2=Removed

% Define the 1st order Moore neighborhood, i.e. the 8 nearest neighbors
neigh = [-1 -1; 0 -1; 1 -1; 1 0; 1 1; 0 1; -1 1; -1 0];

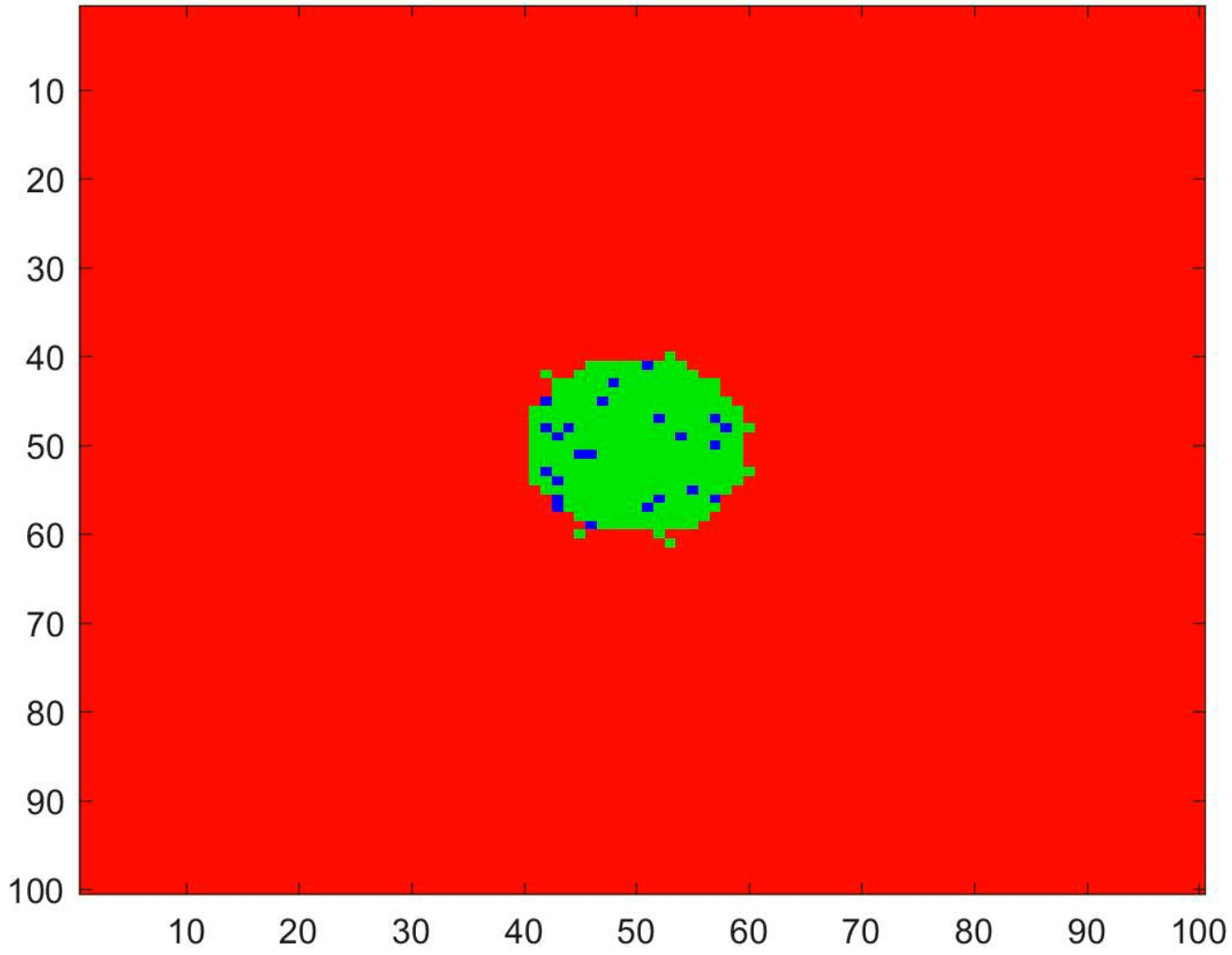
% Main loop, iterating the time variable, t
for t=1:1000

    % Iterate over all cells in grid x, for index i=1..N and j=1..N
    for i=1:N
        for j=1:N

            % Iterate over the neighbors and spread the disease
            for k=1:8
                i2 = i+neigh(k, 1);
                j2 = j+neigh(k, 2);
                % Check that the cell is within the grid boundaries
                if ( i2>=1 && j2>=1 && i2<=N && j2<=N )
                    %[DO SOMETHING]
                end
            end

        end
    end

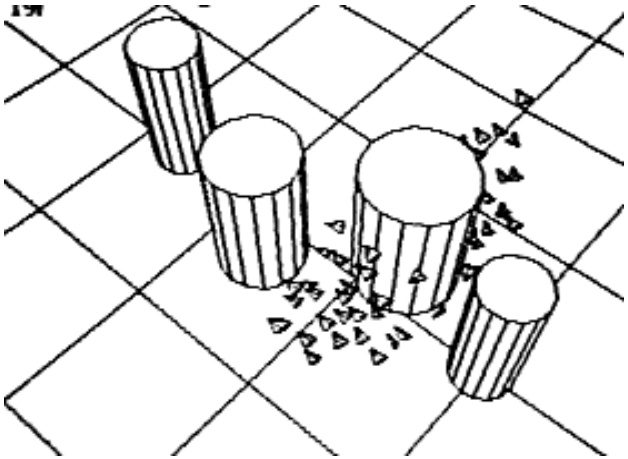
end
```



Susceptible Infected Recovered

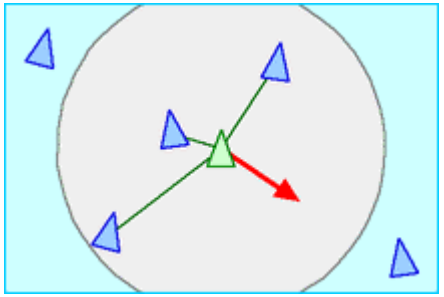
Boids (Reynolds,1986)

- A '*boid*' is a simulated individual particle or object moving around in a virtual space.
- Trivia: Used in 1992 Tim Burton film *Batman Returns*

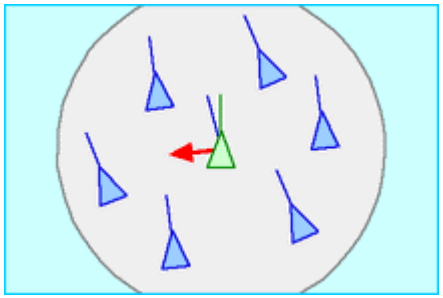




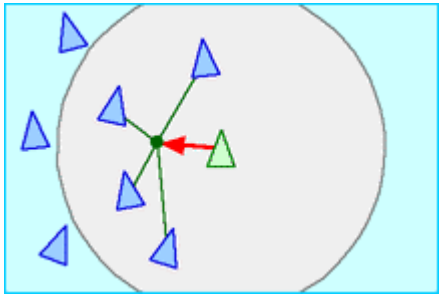
Boids (Reynolds,1986)



Separation: steer to avoid crowding local flockmates



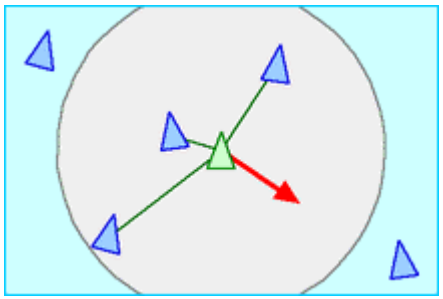
Alignment: steer towards the average heading of local flockmates



Cohesion: steer to move toward the average position of local flockmates

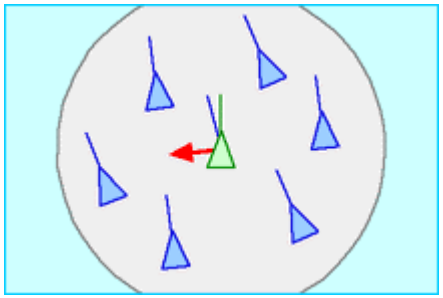
http://www.red3d.com/cwr/boids/

Boids (Reynolds,1986)



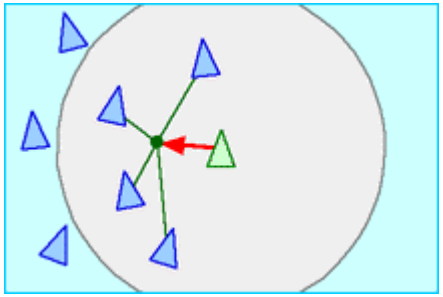
Separation: steer to avoid crowding local flockmates

$$\vec{f}_i^{\,separation} = \alpha \sum_{j=1}^{k} \frac{\vec{x}_i - \vec{x}_j}{\left\| \vec{x}_i - \vec{x}_j \right\|} \Theta\left(\left\| \vec{x}_i - \vec{x}_j \right\| - r \right)$$



Alignment: steer towards the average heading of local flockmates

$$\vec{f}_i^{\,alignment} = \beta \frac{1}{k} \sum_{j=1}^{k} \left(\dot{\vec{x}}_j - \dot{\vec{x}}_i \right)$$



Cohesion: steer to move toward the average position of local flockmates

$$\vec{f}_i^{\,cohesion} = \gamma \frac{\left(\frac{1}{k} \sum_{j=1}^{k} \vec{x}_j \right) - \vec{x}_i}{\left\| \left(\frac{1}{k} \sum_{j=1}^{k} \vec{x}_j \right) - \vec{x}_i \right\|}$$

Boids (Reynolds,1986)

$$\vec{f}_i^{\,separation} = \alpha \sum_{j=1}^{k} \frac{\vec{x}_i - \vec{x}_j}{\left\| \vec{x}_i - \vec{x}_j \right\|} \Theta\left(\left\| \vec{x}_i - \vec{x}_j \right\| - r \right)$$

$$\vec{f}_i^{\,alignment} = \beta \frac{1}{k} \sum_{j=1}^{k} \left(\dot{\vec{x}}_j - \dot{\vec{x}}_i \right)$$

$$\vec{f}_i^{\,cohesion} = \gamma \frac{\left(\frac{1}{k} \sum_{j=1}^{k} \vec{x}_j \right) - \vec{x}_i}{\left\| \left(\frac{1}{k} \sum_{j=1}^{k} \vec{x}_j \right) - \vec{x}_i \right\|}$$

$$\ddot{\vec{x}}_i = \vec{f}_i^{\,separation} + \vec{f}_i^{\,alignment} + \vec{f}_i^{\,cohesion}$$

